

OFICINA BACKEND · DIA 1 DE 3

API com Node.js do zero ao frontend

Conectando sua primeira API Node.js com um frontend em HTML, CSS e JavaScript.

O que é backend?

02

03

Objetivo desta aula

Entender o papel do backend, criar um servidor Node com Express, expor rotas GET e conectar esses dados em uma página HTML real usando fetch.

00–10 min

Abertura

Contexto, motivação e o que vamos construir hoje.

10–25 min

Setup

Instalar dependências, criar a pasta backend e subir o servidor.

25–65 min

Rotas e API

Criar os endpoints GET, testar no navegador e entender JSON.

65–95 min

Frontend + fetch

Conectar a página HTML ao backend com fetch e renderizar dados.

95–105 min

Desafio + recap

Mini desafio individual e fechamento do ciclo completo.

- Importante: ao final do dia cada aluno terá uma página web lendo dados de uma API Node.js real.

Você já sabe fazer...

- Criar páginas HTML com estrutura semântica.
- Estilizar com CSS e criar layouts responsivos.
- Adicionar interatividade com JavaScript — eventos, DOM, condicionais.

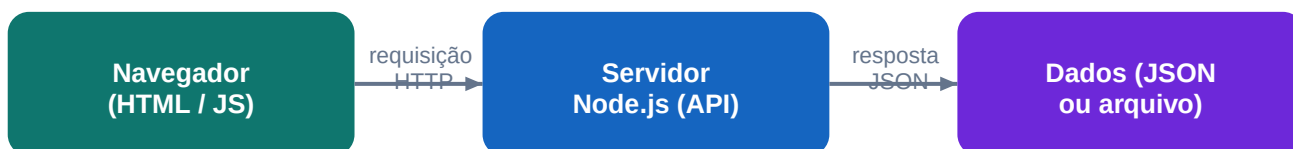
O que ainda não tínhamos...

- Dados que persistem entre sessões (não somem ao recarregar).
- Dados compartilhados entre usuários diferentes.
- Lógica de negócio segura (validação, regras que o usuário não pode burlar).
- Integração com banco de dados.



O backend é a camada que resolve isso. Ele recebe pedidos do frontend, processa as regras e devolve os dados prontos.

A comunicação acontece via protocolo HTTP. O navegador faz uma requisição e o servidor responde com dados.



O que é JSON?

JSON (JavaScript Object Notation) é o formato de dados que backend e frontend usam para se entender. É texto puro, simples de ler e enviar pela rede.

```
// Exemplo de resposta JSON da API de cursos
{
  "id": 1,
  "nome": "HTML e CSS",
  "nivel": "iniciante"
}
```

HTTP — verbos que precisamos conhecer

GET

Buscar dados (listar cursos, carregar perfil)

POST

Criar algo novo (cadastrar tarefa, enviar recado)

PUT

Atualizar um item existente (editar tarefa)

DELETE

Remover um item (apagar tarefa)

& Pré-requisito: Node.js LTS instalado. Teste no terminal: `node -v` e `npm -v`

Passo 1 — Criar a pasta e iniciar o projeto

```
// No terminal, dentro da pasta oficina_back/dia1-api-basica/backend:  
npm init -y           // cria o package.json automaticamente  
npm install express cors // instala as dependências  
npm install -D nodemon // reinicia o servidor ao salvar
```

Passo 2 — Ajustar o package.json para usar nodemon

```
// package.json - trecho de "scripts"  
"scripts": {  
  "dev": "nodemon server.js", // reinicia ao salvar  
  "start": "node server.js"  // para produção  
}
```

Passo 3 — Subir o servidor

```
npm run dev  
  
// Saída esperada no terminal:  
// [nodemon] starting `node server.js`  
// Servidor rodando em http://localhost:3333
```

- Com isso o servidor já está no ar. Qualquer edição que você salvar reinicia automaticamente graças ao nodemon.

Entendendo o Express e a estrutura do servidor

CONCEITO + CÓDIGO

Oficina Backend Node.js | Dia 1

Express é um framework minimalista para criar servidores HTTP com Node.js. Com ele, registramos rotas que respondem a requisições do tipo GET, POST, PUT e DELETE.

```
// server.js – estrutura comentada linha a linha
const express = require('express'); // importa o framework
const cors    = require('cors');    // libera acesso do frontend

const app = express(); // cria a aplicação
const PORT = 3333;

// Middlewares – processam TODA requisição antes das rotas
app.use(cors());           // permite que o browser acesse a API
app.use(express.json());   // lê o body JSON das requisições POST/PUT

// Rota: GET /
// Chamada quando o browser acessa http://localhost:3333/
app.get('/', (req, res) => {
  res.json({ mensagem: 'API funcionando!' });
  //      !' envia resposta em formato JSON
});

// Sobe o servidor e fica escutando conexões
app.listen(PORT, () => {
  console.log(`Servidor em http://localhost:${PORT}`);
});
```

Dois objetos que você vai usar em toda rota

req

Request — contém tudo que chegou do cliente: body, params, headers.

res

Response — usado para enviar a resposta: res.json(), res.status().

Vamos criar três rotas GET no mesmo `server.js`. Cada uma devolve dados em JSON. Abra o arquivo, copie e rode.

```
// GET / - rota raiz: boas-vindas
app.get('/', (req, res) => {
  res.json({
    mensagem: 'API da oficina funcionando.',
    dica:      'Acesse /status e /cursos'
  });
});

// GET /status - health check: útil para saber se a API está viva
app.get('/status', (req, res) => {
  res.json({
    status: 'ok',
    horario: new Date().toISOString() // data/hora atual do servidor
  });
});

// GET /cursos - retorna array de cursos em memória
const cursos = [
  { id: 1, nome: 'HTML e CSS',          nivel: 'iniciante' },
  { id: 2, nome: 'JavaScript para Web', nivel: 'iniciante' },
  { id: 3, nome: 'Backend com Node.js', nivel: 'iniciante' },
];

app.get('/cursos', (req, res) => {
  res.json(cursos); // envia o array inteiro como JSON
});
```

Como testar sem escrever uma linha de frontend

- Abra o navegador e acesse `http://localhost:3333/`
- Acesse `http://localhost:3333/status` — deve mostrar `status: ok`
- Acesse `http://localhost:3333/cursos` — deve mostrar o array
- *Dica: instale a extensão "JSON Viewer" no Chrome para ver o JSON formatado.*

Toda resposta HTTP carrega um código numérico de 3 dígitos. Aprender a ler esses códigos é o principal atalho para debugar APIs.

200 OK

Requisição bem-sucedida. Dados retornados normalmente.

201 Created

Recurso criado com sucesso (usar em POST que cria algo).

400 Bad Request

Dados enviados pelo cliente são inválidos ou faltam campos.

401 Unauthorized

Falta autenticação (token ausente ou inválido).

404 Not Found

Rota ou recurso não existe no servidor.

500 Internal Server Error

Erro inesperado no backend — olhe o terminal do servidor.

```
// Como definir o status em uma rota
app.get('/rota-prottegida', (req, res) => {
  const autenticado = false; // simulação
  if (!autenticado) {
    return res.status(401).json({ erro: 'Acesso negado' });
    //      !' define o código antes do .json()
  }
  res.json({ dados: 'conteúdo secreto' }); // 200 é o padrão
});
```


`fetch()` é nativo no browser. Com ele fazemos requisições HTTP diretamente do JavaScript, sem instalar nada.

Estrutura base de um fetch com tratamento de erro

```
// script.js no frontend
async function carregarCursos() {
  try {
    const resposta = await fetch('http://localhost:3333/cursos');
    // !' faz a requisição GET

    if (!resposta.ok) {
      // resposta.ok é false para status 4xx e 5xx
      throw new Error(`Falha HTTP: ${resposta.status}`);
    }

    const cursos = await resposta.json();
    // !' converte o texto JSON em objeto/array JavaScript

    renderizarCursos(cursos); // função que atualiza o DOM

  } catch (erro) {
    console.error('Erro ao buscar cursos:', erro);
    mostrarMensagem('Não foi possível conectar à API.');
```

Renderizando os dados no DOM

```
function renderizarCursos(cursos) {
  const lista = document.getElementById('listaCursos');
  lista.innerHTML = ''; // limpa antes de re-renderizar

  cursos.forEach((curso) => {
    const item = document.createElement('li');
    item.textContent = `${curso.nome} (${curso.nivel})`;
    lista.appendChild(item);
  });
}
```

Por que usar `async/await`?

`fetch` é assíncrono — a resposta não chega na mesma hora. `async/await` faz o código esperar sem travar a página.

```
// 'L' Errado – tenta usar a resposta antes de ela chegar
const resposta = fetch('http://localhost:3333/cursos');
console.log(resposta); // imprime Promise {<pending>}, não os dados!

// 'C' Correto – espera a Promise resolver com await
const resposta = await fetch('http://localhost:3333/cursos');
const dados = await resposta.json();
console.log(dados); // [ { id: 1, nome: 'HTML e CSS', ... } ]
```

Quando o CORS bloqueia a requisição

Se você ver no console "CORS policy blocked" — significa que o backend não está liberando o acesso do browser. Solução: garantir que `app.use(cors())` está no `server.js` ANTES das rotas.

Testando se a API está no ar antes de apontar o frontend

- Abra o terminal e confirme que `npm run dev` está rodando.
- Acesse `http://localhost:3333/cursos` no próprio navegador.
- Se aparecer JSON: API está funcionando. Agora conecte o frontend.
- Se der erro de porta em uso, mude `PORT = 3333` para `3334` no `server.js`.

```
<!-- index.html – estrutura mínima -->
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8" />
  <title>Dia 1 - Frontend + API</title>
  <link rel="stylesheet" href="./style.css" />
</head>
<body>
  <main class="container">
    <h1>Cursos da API</h1>
    <button id="btnCarregar">Carregar cursos</button>
    <p id="mensagem"></p>
    <ul id="listaCursos"></ul>
  </main>
  <script src="./script.js"></script>
</body>
</html>
```

```
// script.js – versão final com feedback visual
const btnCarregar = document.getElementById('btnCarregar');
const mensagem = document.getElementById('mensagem');
const listaCursos = document.getElementById('listaCursos');

async function carregarCursos() {
  mensagem.textContent = 'Carregando cursos...';
  listaCursos.innerHTML = '';

  try {
    const resposta = await fetch('http://localhost:3333/cursos');
    if (!resposta.ok) throw new Error(`Status: ${resposta.status}`);

    const cursos = await resposta.json();
    cursos.forEach((c) => {
      const li = document.createElement('li');
      li.textContent = `${c.nome} – ${c.nivel}`;
      listaCursos.appendChild(li);
    });
    mensagem.textContent = `${cursos.length} curso(s) carregado(s).`;
  } catch (err) {
    mensagem.textContent = 'Não foi possível conectar à API.';
  }
}

btnCarregar.addEventListener('click', carregarCursos);
```

Cannot find module 'express'

Causa: Dependências não foram instaladas.

' **Rode npm install dentro da pasta backend.**

EADDRINUSE – porta 3333 já está em uso

Causa: Outro processo está ocupando a porta.

' **Feche terminais com servidor rodando ou mude PORT no server.js.**

CORS blocked no console do browser

Causa: app.use(cors()) não está no server.js, ou foi colocado depois das rotas.

' **Verifique se cors() está antes de qualquer app.get/post.**

fetch failed – Failed to fetch

Causa: A API não está rodando ou a URL está errada.

' **Abra http://localhost:3333 no browser antes de testar o frontend.**

SyntaxError: Unexpected token < em JSON

Causa: A rota retornou HTML (geralmente página de erro 404).

' **Verifique o caminho da rota no fetch e no server.js.**

!9 Você tem 20 minutos. Trabalhe individualmente ou em dupla. O instrutor circula para apoiar.

O que você vai construir

- Criar o endpoint GET /instrutores no server.js com pelo menos 2 instrutores.
- Cada instrutor deve ter: id, nome, especialidade.
- No frontend, adicionar um segundo botão "Carregar instrutores".
- Ao clicar, buscar o endpoint e renderizar os instrutores na tela.
- Tratar o erro caso a API esteja fora do ar (mensagem amigável).

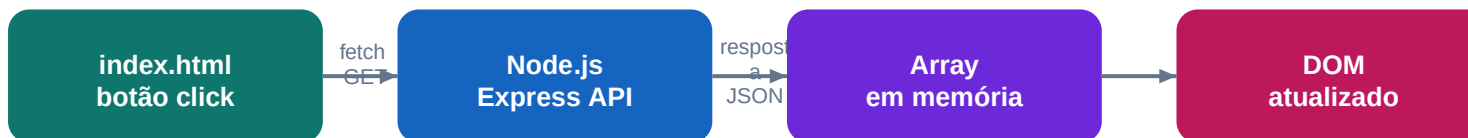
Estrutura esperada do endpoint

```
// Adicione ao server.js – abaixo dos cursos
const instrutores = [
  { id: 1, nome: 'Ana', especialidade: 'HTML e CSS' },
  { id: 2, nome: 'Bruno', especialidade: 'JavaScript' },
];

app.get('/instrutores', (req, res) => {
  res.json(instrutores);
});
```

Checklist de entrega

- Abrir <http://localhost:3333/instrutores> no browser e ver o JSON.
- Clicar no botão do frontend e ver os nomes na tela.
- Parar a API e ver a mensagem de erro aparecer no frontend.



Conquistas do dia

- ' Servidor Node.js funcionando com Express.
- ' Três rotas GET retornando JSON válido.
- ' Frontend conectado à API com fetch e async/await.
- ' Tratamento de erro com try/catch e feedback ao usuário.
- ' Conhecimento do ciclo completo: request !' response !' DOM.

O que vem no Dia 2

- Criar, listar, editar e remover dados — o famoso CRUD.
- Persistir dados em arquivo JSON (não perder ao reiniciar).
- Enviar dados do frontend para o backend com POST e PUT.